

Job Sequencing with Deadlines

Algorithm JSGreedy (J,D, P, n)

Input:

J → an array of jobs

D → an array of deadlines associated with jobs in J

P → an array of profit associated with jobs in J

Output:

S → Solution array containing scheduled jobs

profit → maximum profit achieved by completing jobs in S

Start:

// sort in non-increasing order of profit values

J,D,P ← Sort(J,D,P)

sol_array_size ← min (n, max(D))

profit ← 0

for i ← 1 to sol_array_size do

 S[i] = EMPTY

for i ← 1 to n do

 k ← min(n, D[i])

 while k >= 1 do

 if S[k] is EMPTY then

 S[k] ← J[i]

 profit ← profit + P[i]

 break

 k ← k-1

print (profit)

return S

End:

Job Sequencing with deadline using Disjoint set data structure

Find and Union functions :

function find(s):

 if s ≠ parent[s]:

 parent[s] = find(parent[s]) // Path Compression

 return parent[s]

function union(u, v):

 parent[v] = u

Algorithm FJS(J, D, P, n)

Start:

```
// Initialize variables:
maxDeadline = maximum (D[i])
profit = 0
m = min(n,maxDeadline) // Number of slots

// initializeDisjointSet:
parent = array of integers of size m+1
for i = 0 to m:
    parent[i] = i

// sort J, D, P in non-increasing order of profit
J, D, P = sort(J, D, P)

// apply greedy technique
k=0
for i = 1 to n:
    availableSlot = find(min(n, D[i]))
    if availableSlot > 0:
        k=k+1
        S[k] = i
        profit = profit + P[i]
        union(find(availableSlot - 1), availableSlot)

print(profit)
return S
```

End:

Analysis:

Let's analyze the time complexity of the provided pseudo-code for the Job Sequencing with Deadlines algorithm. The key operations contributing to the time complexity are the sorting of jobs and the disjoint set operations.

Let:

- n be the number of jobs,
- m be the maximum deadline among all jobs.

The time complexity can be broken down as follows:

1. **Sorting:** The `sort` function has a time complexity of $O(n \log n)$ in the worst case. Sorting the jobs based on their profits is the dominating factor.
2. **Disjoint Set Operations:**
 - **Initialization:** The `initializeDisjointSet` function initializes the disjoint set data structure. This operation takes $O(m)$ time since the size of the disjoint set array is determined by the maximum deadline among the jobs.

- **Find (with Path Compression):** The `find` function has an amortized time complexity of $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function. In practical terms, this is almost constant time due to the rapid convergence of the Ackermann function.

- **Union:** The `union` function has a constant time complexity.

3. **Finding Maximum Deadline:** The `max` function iterates through all jobs once, taking $O(n)$ time.

4. **Loop:** The loop iterates through all jobs once, and each iteration involves disjoint set operations. Therefore, this part contributes $O(n\alpha(n))$ to the time complexity.

Overall, the dominant term in the time complexity is the sorting operation:

$$T(n,m)=O(n\log n)+O(m)+O(n\alpha(n))$$

In practical scenarios, the disjoint set operations can be considered effectively constant due to the properties of the Ackermann function. Therefore, the overall time complexity is often expressed as **$O(n\log n)$** .

Minimum Spanning Tree

Assume that we have a connected, undirected graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$, and we wish to find a minimum spanning tree for G . The two algorithms we consider here use a greedy approach to the problem, although they differ in how they apply this approach.

This greedy strategy is captured by the following generic method, which grows the minimum spanning tree one edge at a time. The generic method manages a set of edges A , maintaining the following loop invariant:

Prior to each iteration, A is a subset of some minimum spanning tree.

At each step, we determine an edge (u,v) that we can add to A without violating this invariant, in the sense that $A \cup \{(u,v)\}$ is also a subset of a minimum spanning tree. We call such an edge a **safe edge** for A , since we can add it safely to A while maintaining the invariant.

GENERIC-MST(G, w)

```
1  $A = \emptyset$ ;  
2 while  $A$  does not form a spanning tree  
3     find an edge  $(u,v)$  that is safe for  $A$   
4      $A = A \cup \{(u,v)\}$   
5 return  $A$ 
```

Kruskal's Algorithm

MST-KRUSKAL (G, w)

```
1  $A = \emptyset$   
2 for each vertex  $v \in G.V$   
3     MAKE-SET ( $v$ )  
4 sort the edges of  $G.E$  into nondecreasing order by weight  $w$   
5 for each edge  $(u,v) \in G.E$ , taken in nondecreasing order by weight  
6     if FIND-SET ( $u$ )  $\neq$  FIND-SET ( $v$ )  
7          $A = A \cup \{(u,v)\}$   
8     UNION ( $u,v$ )  
9 return  $A$ 
```

Let's analyze the time complexity step by step:

- **Initialize Sets:**
 - The MAKE - SET operation is performed for each vertex which takes $O(1)$ time. In the worst case, this is $O(V)$, as the number of vertices is V .
- **Sort Edges:**
 - Sorting the edges by weight using a standard sorting algorithm takes $O(E \lg E)$ time, where E is the number of edges.
- **Iterate Through Sorted Edges:**

- The loop iterates through the sorted edges, and for each edge, it performs disjoint set operations which takes $O(\alpha(V))$ time, where $\alpha(V)$ is the inverse Ackermann function. This is due to the use of disjoint-set operations (**FIND-SET** and **UNION**), which are optimized with path compression (collapsing find) and union by rank (similar to weighted union).
- Because we assume that G is connected, we have $|E| \geq |V| - 1$, and so the disjoint -set operations take $O(E \alpha(V))$ time.
- In practice, the disjoint-set operations can be considered nearly constant due to the rapid convergence of the Ackermann function. Still theoretically $\alpha(V)$ has upper bound of $O(\lg V)$.
- Hence, $\alpha(|V|) = O(\lg V) = O(\lg E)$.

The overall time complexity can be expressed as:

$$T(V,E) = O(V) + O(E \lg E) + O(E \alpha(V)) = O(V) + O(E \lg E) + O(E \lg E)$$

Therefore, the overall time complexity is often simplified to:

$$T(V,E) = O(E \lg E)$$

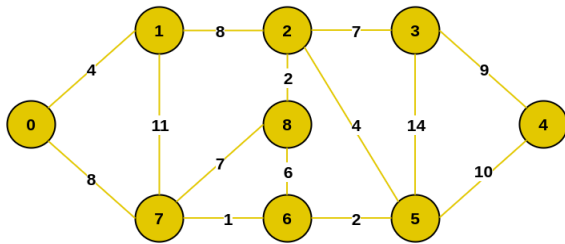
Also it is to be observed that $|E| < |V|^2$, we have $\lg |E| = O(\lg V)$, and so we can restate the running time of Kruskal's algorithm as **$O(E \lg V)$** .

Prim's Algorithm

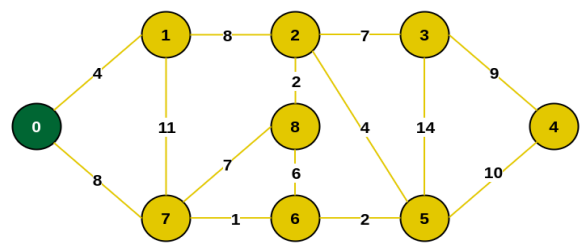
MST-PRIM (G, w, r)

1. **for** each $u \in G.V$
2. $u.key = \infty$
3. $u.\pi = \text{NIL}$
4. $r.key = 0$
5. $Q = G.V$
6. **while** $Q \neq \emptyset$
7. $u = \text{EXTRACT-MIN}(Q)$
8. **for** each $v \in G.\text{Adj}[u]$
9. if $v \in (Q)$ and $w(u,v) < v.key$
10. $v.\pi = u$
11. $v.key = w(u,v)$

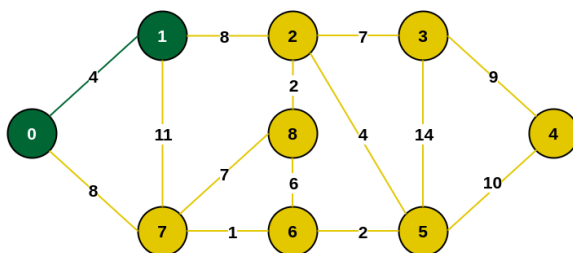
Example:



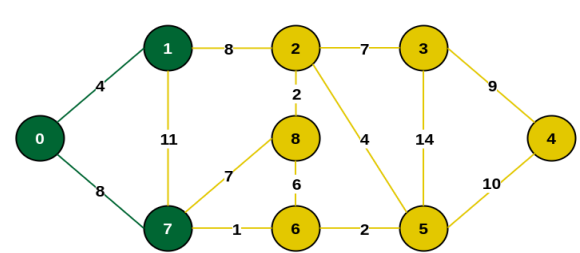
Example of a Graph



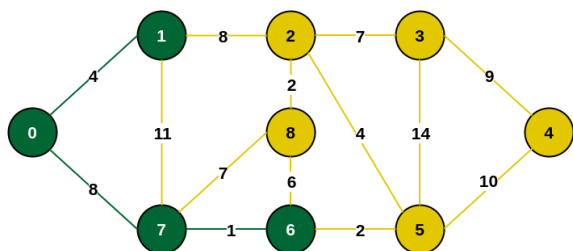
Select an arbitrary starting vertex. Here we have selected 0



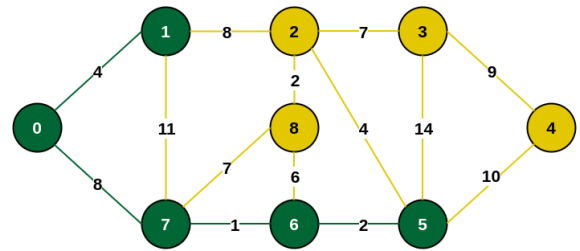
Minimum weighted edge from MST to other vertices is 0-1 with weight 4



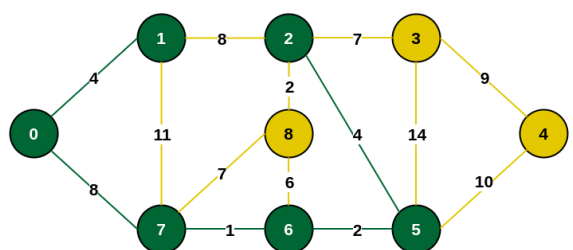
Minimum weighted edge from MST to other vertices is 0-7 with weight 8



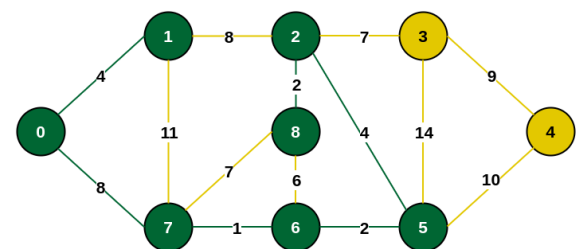
Minimum weighted edge from MST to other vertices is 7-6 with weight 1



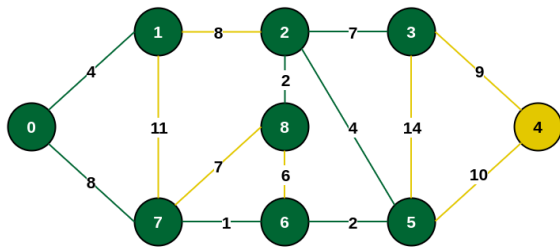
Minimum weighted edge from MST to other vertices is 6-5 with weight 2



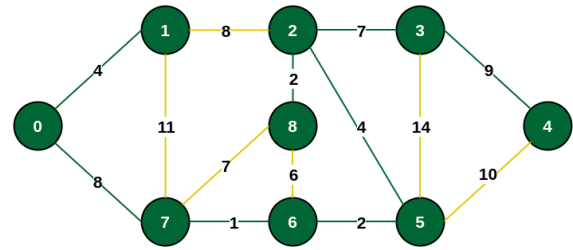
Minimum weighted edge from MST to other vertices is 5-2 with weight 4



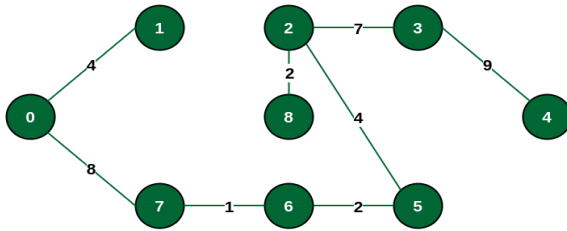
Minimum weighted edge from MST to other vertices is 2-8 with weight 2



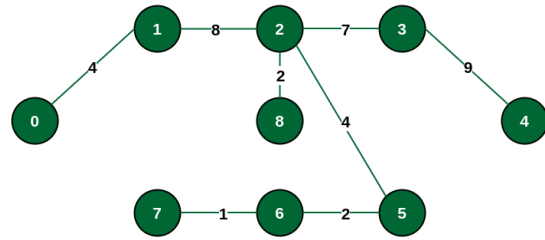
Minimum weighted edge from MST to other vertices is 2-3 with weight 7



Minimum weighted edge from MST to other vertices is 3-4 with weight 9



The final structure of MST



Alternative MST structure

Analysis of Each Step

1. Initialization (Lines 1–3):

- Loop over all vertices $u \in G.V$ to initialize $u.key = \infty$ and $u.\pi = \text{NIL}$.
- Time Complexity: $O(V)$, where V is the number of vertices.

2. Priority Queue Initialization (Line 5):

- Create a priority queue Q using min-heap for all vertices $G.V$
- Time Complexity: $O(V)$ using the **BUILD-MIN-HEAP** procedure

3. Main While Loop (Lines 6–11):

- The loop runs until all vertices are processed, i.e., Q becomes empty.
- The loop iterates V times because there are V vertices in Q .

Key operations in the loop:

• **EXTRACT-MIN (Line 7):**

- Extracts the vertex u with the smallest $u.key$ from the priority queue Q .
- Time Complexity depends on the data structure used for the priority queue:
 - **Binary Heap:** $O(\log V)$ per operation.
 - **Fibonacci Heap:** $O(\log V)$ amortized time.
- Since this operation is repeated V times, the total time for EXTRACT-MIN is:
 - **Binary Heap:** $O(V \log V)$
 - **Fibonacci Heap:** $O(V)$ amortized.

• **For Each Edge in the Adjacency List (Lines 8–11):**

- For each vertex u , iterate over its adjacent vertices $v \in G.\text{Adj}[u]$.

- Check if v is still in the priority queue and whether the edge weight $w(u,v)$ is smaller than $v.key$. If so:
 - Update $v.key$ and $v.\pi$.
 - Decrease the key of v in the priority queue.
- The **DECREASE-KEY** operation is performed for each edge (u,v) in the graph.

Time Complexity of **DECREASE-KEY**:

- **Binary Heap**: $O(\log V)$ per operation.
- **Fibonacci Heap**: $O(1)$ amortized.

Since the **for** loop iterates over all edges in the graph, the total time for edge relaxations (over all iterations of while loop) is:

- **Binary Heap**: $O(E \log V)$
- **Fibonacci Heap**: $O(E)$ amortized.

Total Time Complexity

Combining all the components:

1. Initialization: $O(V)$
2. **EXTRACT-MIN** (for V vertices):
 - Binary Heap: $O(V \log V)$
 - Fibonacci Heap: $O(V \log V)$ amortized
3. Edge Relaxations (for E edges):
 - Binary Heap: $O(E \log V)$
 - Fibonacci Heap: $O(E)$ amortized

Final Time Complexity:

- Using a **Binary Heap**: $O(V \log V + E \log V) = O(E \log V)$ since E dominates V for connected graphs.
- Using a **Fibonacci Heap**: $O(V \log V + E)$.

Conclusion:

- With a **Binary Heap**, the time complexity of Prim's algorithm is **$O(E \log V)$** .
- With a **Fibonacci Heap**, the time complexity is **$O(V \log V + E)$** , which is more efficient for dense graphs where E is large compared to V .

Single Source Shortest Path

Dijkstra's Algorithm:

Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are nonnegative. Therefore, we assume that $w(u,v) \geq 0$ for each edge $(u,v) \in E$.

Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined. The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S , and relaxes all edges leaving u . In the following implementation, we use a min-priority queue Q of vertices, keyed by their d values.

DIJKSTRA (G, w, s)

```

1 INITIALIZE-SINGLE-SOURCE ( $G, s$ )
2  $S = \emptyset$ 
3  $Q = G.V$ 
4 while  $Q \neq \emptyset$ 
5    $u = \text{EXTRACT-MIN}(Q)$ 
6    $S = S \cup \{u\}$ 
7   for each vertex  $v \in G.\text{Adj}[u]$ 
8     RELAX( $u, v, w$ )
    
```

INITIALIZE-SINGLE-SOURCE (G, s)

```

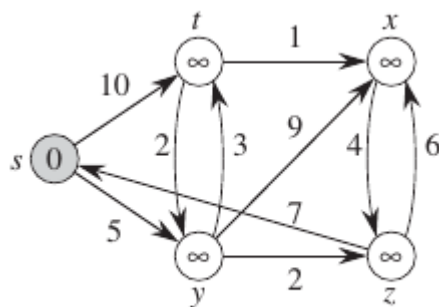
1 for each vertex  $v \in G.V$ 
2    $v.d = \infty$ 
3    $v.\pi = \text{NIL}$ 
4  $s.d = 0$ 
    
```

RELAX (u, v, w)

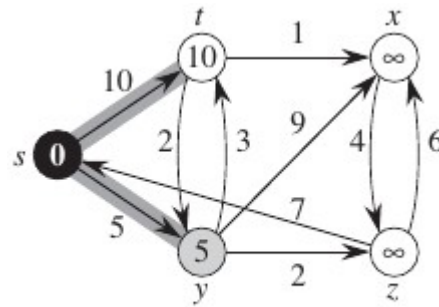
```

1 if  $v.d > u.d + w(u, v)$ 
2    $v.d = u.d + w(u, v)$ 
3    $v.\pi = u$ 
    
```

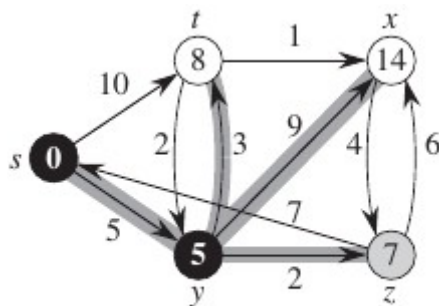
Example:



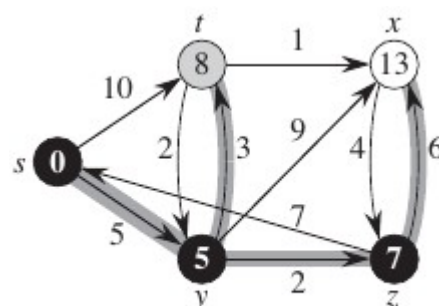
(a)



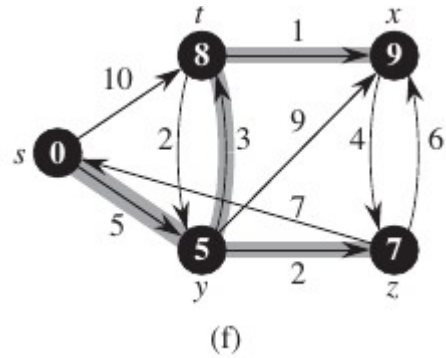
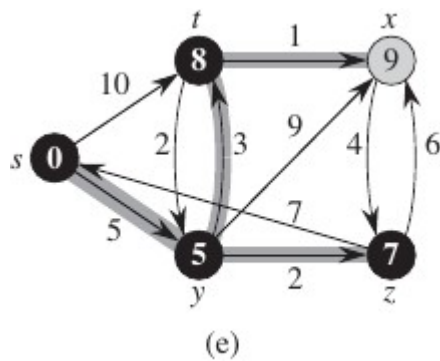
(b)



(c)



(d)



The execution of Dijkstra's algorithm. The source s is the leftmost vertex. The shortest-path estimates appear within the vertices, and shaded edges indicate predecessor values. Black vertices are in the set S , and white vertices are in the min-priority queue $Q = V - S$. **(a)** The situation just before the first iteration of the **while** loop of lines 4–8. The shaded vertex has the minimum d value and is chosen as vertex u in line 5. **(b)–(f)** The situation after each successive iteration of the **while** loop. The shaded vertex in each part is chosen as vertex u in line 5 of the next iteration. The d values and predecessors shown in part **(f)** are the final values.

Analysis of Each Step

1. Initialization (**INITIALIZE - SINGLE - SOURCE**):

- This part initializes the distance ($v.d$) of all vertices to infinity (∞), except for the source vertex s where $s.d=0$.
- Time Complexity: $O(V)$, where V is the number of vertices.

2. Priority Queue Initialization (**Line 3**):

- Create a priority queue Q using min-heap for all vertices $G.V$
- Time Complexity: $O(V)$ using the BUILD-MIN-HEAP procedure

3. Outer **while** Loop (**line 4–5**): The **while** loop runs $|V|$ times, once for each vertex, because each vertex is extracted exactly once from the priority queue Q .

- **EXTRACT - MIN(Q) Operation** (line 5): Using a **min-heap**, each EXTRACT - MIN operation takes $O(\log V)$ time.

4. Relaxation of Edges (**line 7–8**): For each vertex u , all adjacent edges (u,v) are relaxed:

- Each edge (u,v) is relaxed **at most once** because we only process a vertex u once.
- The total number of edge relaxations is $O(E)$, where E is the number of edges.
- Each relaxation operation may involve a **DECREASE-KEY** operation on the priority queue, which takes $O(\log V)$ time.

Combining the Time Complexity

1. Initialization: $O(V)$
2. EXTRACT-MIN(Q):

- Runs $|V|$ times (once for each vertex).
- Each operation takes $O(\log V)$ time.
- Total time: **$O(V \log V)$** .

3. Relaxation and DECREASE - KEY:

- Each edge (u,v) is relaxed once.
- Relaxation involves updating the key in the priority queue (using DECREASE - KEY), which takes $O(\log V)$ time.
- Total time: **$O(E \log V)$** .

Overall Time Complexity (Binary heap)

Combining all components:

- **Initialization:** $O(V)$
- **Priority Queue Operations:** $O(V \log V)$ for EXTRACT - MIN
- **Edge Relaxations:** $O(E \log V)$ for DECREASE - KEY

Thus, the total time complexity using a binary heap is: **$O((V+E) \log V)$**

Explanation with Binary Heap

- If the graph is **sparse** (i.e., $E = O(V)$), the time complexity simplifies to **$O(V \log V)$** .
- If the graph is **dense** (i.e., $E = O(V^2)$), the time complexity simplifies to **$O(V^2 \log V)$** .

Overall Time Complexity (Fibonacci heap):

Combining all components:

- **Initialization:** $O(V)$
- **Priority Queue Operations:** $O(V \log V)$ for EXTRACT - MIN
- **Edge Relaxations:** $O(E)$ for DECREASE - KEY

Thus, the total time complexity using a Fibonacci heap is: **$O(V \log V + E)$**

Explanation with Fibonacci Heap

- If the graph is **sparse** (i.e., $E = O(V)$), the time complexity simplifies to **$O(V \log V)$** .
- If the graph is **dense** (i.e., $E = O(V^2)$), the time complexity simplifies to **$O(V^2)$** .

MinVertexCover (G) :

$Q = G.V$ //create a max heap based on the degree of the vertex

$S = \phi$ //vertex cover set

$e = |G.E|$ // number of edges

while ($e > 0$)

$u = \text{extract-max-heap}(Q)$

$S = S \cup \{u\}$

 for (each $v \in \text{Adj}(u)$)

 if ($v \in Q$)

$v.\text{degree} = v.\text{degree} - 1$

$e = e - 1$

print(S)